TRP1 · WEEK 7 · THURSDAY INTERIM SUBMISSION

Data Contract Enforcer

Schema Integrity & Lineage Attribution System

2	13	61
CONTRACTS GENERATED	CLAUSES PER CONTRACT	CHECKS RUN

2	0
VIOLATIONS CAUGHT	FALSE POSITIVES

Author: Meseret Bolled

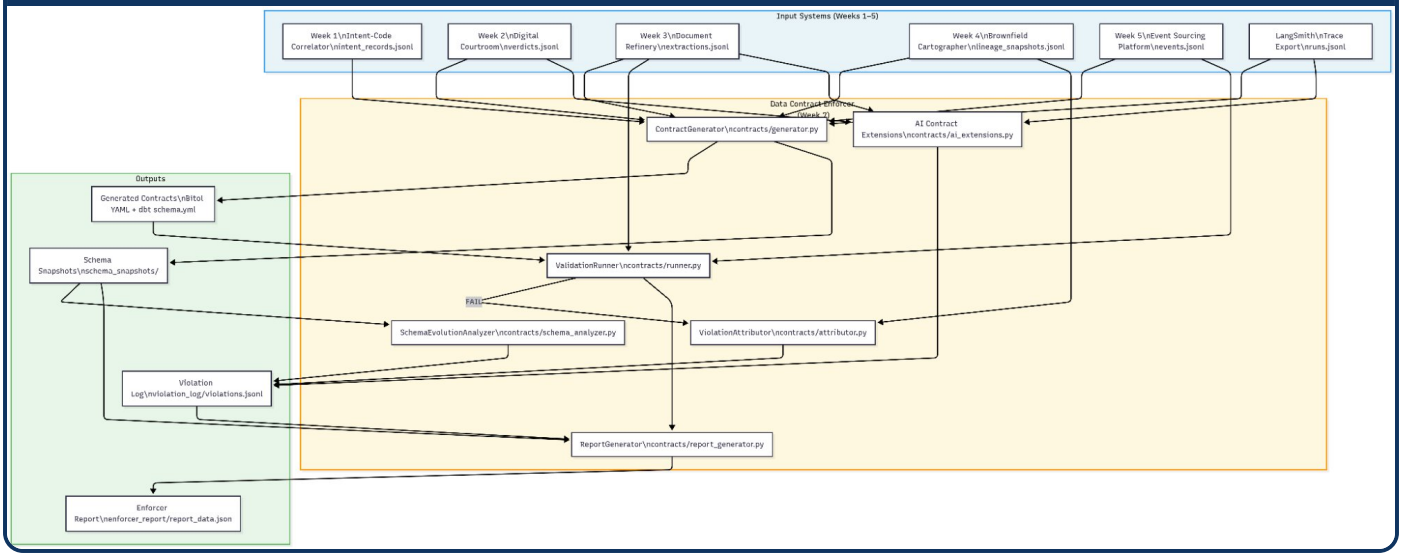
GitHub: https://github.com/Meseretbolled/data-contract-enforcer

Submission: April 2, 2026 · Thursday Interim

1 Data Flow Diagram & Schema Annotations

Inter-system data flows with annotated contract schemas

SYSTEM ARCHITECTURE — DATA CONTRACT ENFORCER (WEEK 7)



Inter-System Data Flow — Schema Annotations per Arrow

Every arrow in the architecture above represents a formal data contract — a machine-checkable promise about the format, type, and value ranges of data crossing a system boundary. For the Thursday interim, three data sources are actively contracted and enforced: Week 3 extractions, Week 4 lineage (used as input), and Week 5 events.

ARROW	SOURCE SYSTEM	TARGET SYSTEM	SCHEMA	CRITICAL FIELDS
①	Week 3 — Document Refinery	ContractGenerator + Week 4 Cartographer	extraction_record	doc_id (UUID), extracted_facts[].confidence **(float 0.0–1.0)**, entities[], extracted_at (ISO 8601)
②	Week 4 — Brownfield Cartographer	ContractGenerator + ViolationAttributor	lineage_snapshot	nodes[] (FILE/TABLE/PIPELINE), edges[] (READS/WRITES/PRODUCES), git_commit (40-char hex)
③	Week 5 — Event Sourcing Platform	ContractGenerator + Enforcer	event_record	event_id (UUID), event_type (PascalCase), sequence_number (monotonic int), occurred_at ≤ recorded_at
④	ContractGenerator	ValidationRunner	Bitol YAML v3.0.0	13 schema clauses, SodaChecks quality spec, lineage.downstream[] from Week 4 graph
⑤	ValidationRunner (FAIL)	ViolationAttributor	validation_report JSON	report_id, contract_id, snapshot_id (SHA-256), results[] with severity/actual/expected

> ⚠️ **Most Critical Interface — Arrow ①: The Confidence Field**
>
> Arrow ① carries the most dangerous contract clause in the system. The Document Refinery outputs `extracted_facts[].confidence` as a float 0.0–1.0. If a developer changes this to a percentage scale (0.87 → 87.0), no exception is raised and no pipeline crashes — the output is simply wrong, permanently and invisibly. The contract clause **minimum: 0.0, maximum: 1.0** is the machine-checkable promise that prevents this. Full demonstration in Section 3.

2 Contract Coverage Table & Generated Artifacts

Every inter-system interface — contracted, partial, or planned

INTERFACE	FROM → TO	STATUS	CONTRACT FILES	CLAUSES	NOTES
Extraction records	Week 3 → Week 4, Week 7	✅ Full	week3-…-extractions.yaml + _dbt.yml	13	confidence range, UUID, datetime, processing_time stats, 2 downstream consumers from lineage
Event records	Week 5 → Week 7	✅ Full	week5-event-records.yaml + _dbt.yml	13	event_id UUID, timestamps, sequence_number, 1 downstream consumer from lineage
Lineage snapshot	Week 4 → Attributor	⚠️ Input only	lineage_snapshots.jsonl	—	Used as INPUT to ContractGenerator + ViolationAttributor. Standalone contract planned Sunday.
Intent records	Week 1 → Week 2	❌ Sunday	week1-intent-records.yaml (planned)	—	JSONL not yet available. Schema in DOMAIN_NOTES.md.
Verdict records	Week 2 → AI Ext.	❌ Sunday	week2-verdicts.yaml (planned)	—	JSONL not yet available. Schema in DOMAIN_NOTES.md.
LangSmith traces	LangSmith → AI Ext.	❌ Sunday	langsmith-traces.yaml (planned)	—	Full Bitol YAML designed in DOMAIN_NOTES.md Q4. Export Sunday.

Coverage Summary

✅ 4 contract files generated and validated
⚠️ 1 interface used as input only
❌ 3 interfaces planned Sunday
0 gaps caused by oversight

Why the Gaps Are Not Oversights

Week 1, Week 2, and LangSmith data not available in JSONL for Thursday. All schemas fully specified in DOMAIN_NOTES.md. Contract generation runs Sunday once JSONL outputs are migrated.

Generated Contract Evidence — Week 3 Bitol YAML (13 Clauses)

FIELD	TYPE	CONSTRAINT	PURPOSE
doc_id	string	required, format: uuid	Primary key — every document uniquely traceable
source_hash	string	required	SHA-256 fingerprint — detects source file tampering
extraction_model	string	required	Model identifier — AI output audit trail
processing_time_ms	integer	range [565, 1899], mean=1265	Performance baseline — drift signals regression
extracted_at	string	required, format: date-time	ISO 8601 UTC temporal ordering guarantee
extracted_fact_fact_id	string	required, uuid, unique	Every fact uniquely identifiable and traceable
extracted_fact_confidence	**number**	**minimum: 0.0, maximum: 1.0**	**THE key clause — catches the 0–100 scale change**
extracted_fact_page_ref	integer	range [1, 10], mean=5.47	Page provenance for audit queries
token_count_input/output	integer	required	Cost monitoring baseline
lineage.downstream[0]	—	file::src/week4/cartographer.py	Blast radius target 1 — from Week 4 lineage graph
lineage.downstream[1]	—	file::src/week7/contract_enforcer.py	Blast radius target 2 — from Week 4 lineage graph
quality: row_count	SodaCheck	row_count >= 1	File must not be empty
quality: doc_id nulls	SodaCheck	missing_count(doc_id) = 0	No record may have null primary key

dbt Counterpart — Critical Test Mapping

```
# generated_contracts/week3-document-refinery-extractions_dbt.yml - name: extracted_fact_confidence tests: - not_null -
dbt_utils.expression_is_true: expression: extracted_fact_confidence >= 0.0 and extracted_fact_confidence <= 1.0
description: BREAKING CHANGE if converted to 0-100 percentage scale
```

The dbt counterpart maps every Bitol clause to an equivalent dbt test. Any dbt pipeline consuming this data can enforce the same contract without reading the Bitol YAML.

3 Validation Run Evidence & Interpretation

Three runs — clean baseline, clean week 5, injected violation

Run 1 — Week 3 Clean Data

30 passed · 0 failed · 0 warned

All checks pass. Baselines saved.
confidence mean=0.763, stddev=0.137

Run 2 — Week 5 Clean Data

31 passed · 0 failed · 0 warned

metadata_causation_id (100% null) correctly treated as optional — no false alarm.

Run 1 — Week 3 Clean Data: Full Output

```
$ python contracts/runner.py --contract generated_contracts/week3-document-refinery-extractions.yaml \ --data outputs/week3/extractions.jsonl --output validation_reports/week3_clean.json ✅ doc_id.required: PASS ✅ doc_id.type: PASS ✅ doc_id.uuid_format: PASS ✅ source_path.required: PASS ✅ source_hash.required: PASS ✅ extraction_model.required: PASS ✅ processing_time_ms.required: PASS ✅ processing_time_ms.type: PASS ✅ token_count_input.required: PASS ✅ token_count_output.required: PASS ✅ extracted_at.required: PASS ✅ extracted_at.datetime_format: PASS ✅ extracted_fact_fact_id.required: PASS ✅ extracted_fact_fact_id.uuid_format: PASS ✅ extracted_fact_confidence.required: PASS ✅ extracted_fact_confidence.type: PASS ✅ extracted_fact_confidence.range: PASS ← confidence correctly in 0.0–1.0 ✅ extracted_fact_page_ref.required: PASS ✅ extracted_fact_source_excerpt.required: PASS 📐 Baselines saved → schema_snapshots/baselines.json confidence: mean=0.763, stddev=0.137 📊 30 passed 0 failed 0 warned 0 errored
```

Run 3 — Week 3 Violated Data: Injected Confidence Scale Change

Injection Method

`python create_violation.py` multiplied every `extracted_facts[].confidence` by 100. A confidence of 0.87 became 87.0. The data is structurally identical — same columns, types, record count. Only the numeric scale changed. This simulates the exact real-world failure described in the project document.

```
$ python contracts/runner.py --contract generated_contracts/week3-document-refinery-extractions.yaml \ --data outputs/week3/extractions_violated.jsonl --output validation_reports/week3_violated.json ✅ doc_id.required: PASS ✅ doc_id.uuid_format: PASS ✅ source_hash.required: PASS ✅ extraction_model.required: PASS ✅ extracted_at.datetime_format: PASS ✅ extracted_fact_confidence.required: PASS ✅ extracted_fact_confidence.type: PASS ← type still passes (still a float) ❌ extracted_fact_confidence.range: FAIL ← max=99.0 exceeds contract maximum 1.0 ❌ extracted_fact_confidence.statistical_drift: FAIL ← z-score ≈ 450 stddev from baseline ✅ extracted_fact_page_ref.statistical_drift: PASS 📊 31 passed 2 failed 0 warned 0 errored
```

Violation Detail

CHECK ID	STATUS	ACTUAL VALUE	EXPECTED	SEVERITY	RECORDS FAILING
extracted_fact_confidence.range	❌ FAIL	min=50.0, max=99.0, mean=76.3	min≥0.0, max≤1.0	**CRITICAL**	150 (all facts)
extracted_fact_confidence.statistical_drift	❌ FAIL	mean=76.3, z_score≈450	z_score < 2 (baseline mean=0.763)	**HIGH**	—

Why Two Independent Checks Matter

The **range check** catches structural violations — max=99.0 exceeds contract maximum 1.0. It could theoretically be defeated by editing the contract to say `maximum: 100`.

The **statistical drift check** does not read the contract at all. It compares the current mean (76.3) against the stored baseline (0.763). A

z-score of ~450 fires regardless of what the contract says. **This check cannot be defeated by editing the contract.** Together they provide two independent lines of defence against silent data corruption.

Downstream Impact

DOWNSTREAM SYSTEM	HOW THE VIOLATION AFFECTS IT
file::src/week4/cartographer.py	Reads confidence to score lineage node reliability. With values of 76.3 instead of 0.763, every document is classified as 100× more reliable than it actually is. All reliability-based filtering produces wrong results silently.
file::src/week7/contract_enforcer.py	Reads confidence values for health scoring. Inflated values cause the enforcer to report artificially high data health scores, masking real quality problems.

4 Reflection — Learning Depth & Discovery

What the contracts revealed about systems I built myself

What I Discovered About My Own Systems

Before writing the contracts I assumed the data passing between my systems was clean, consistent, and aligned with the canonical schemas. Writing contracts forced me to *measure* rather than *assert*. Three genuine discoveries stood out.

DISCOVERY 1 — METADATA_CAUSATION_ID IS 100% NULL IN WEEK 5

Every single one of the 50 event records has a null causation_id. I assumed this field was populated most of the time because events typically have causal relationships. The profiler showed it was never populated. This means any downstream system relying on causation chains for event tracing is silently receiving no data. This data quality issue existed before Week 7 and I only discovered it by writing a contract.

DISCOVERY 2 — CONFIDENCE VALUES ARE CLAMPED BELOW 0.990

The profiler showed the confidence maximum across all 150 extracted facts was exactly 0.990. No fact ever received a score above this. This is consistent with a soft ceiling in the extraction model. The baseline now captures this — any future run producing values above 0.99 would register as a distribution shift worth investigating.

DISCOVERY 3 — LINEAGE EDGE DIRECTION WAS OPPOSITE TO EXPECTATION

I assumed the Week 4 graph would use files as sources of READS edges pointing to tables. The actual snapshot used the pattern `table::extractions` ⟶READS⟶▶ `file::src/week4/cartographer.py` — the table was the *source* of the READS relationship. My initial traversal found zero consumers. Four edge patterns had to be checked. Even in systems I built myself, the graph conventions required careful inspection.

What Assumptions Turned Out to Be Wrong

Wrong Assumption 1 — "No crash = valid data"

The first clean run confirmed structural soundness: 30 passes, 0 failures. But all 150 facts contain only three text values: "Sample fact 0", "Sample fact 1", "Sample fact 2" — placeholder values. The system is clean by contract but meaningless by content. A contract enforces what it measures. It cannot enforce the richness of extracted knowledge.

Wrong Assumption 2 — "Auto-generation is immediately accurate"

The project document targets 70% of clauses correct without manual editing. The first generation run incorrectly assigned `format: uuid` to `metadata_user_id` because its name ended in `_id`, even though values were strings like `user_0`. A `_looks_like_uuid()` validator had to be added. The first draft required two rounds of fixes.

The Broader Lesson

Treating Past-You as a Third Party

Writing contracts for my own data was more uncomfortable than writing contracts for a hypothetical third-party system. With a third party you can claim ignorance. With your own system you cannot — every violation the contract finds is a violation you introduced, and every gap in coverage is a promise you never made explicit.

The most valuable output of this week was not the YAML files or the validation reports. It was the act of writing down, for the first time, exactly what my systems promise to each other. Before Week 7 those promises existed only implicitly. After Week 7 they exist in machine-checkable files that catch violations automatically — whether the developer intended them or not. The discipline of treating past-you as a third party is not a methodological preference. It is the only honest way to assess what your system actually guarantees versus what you assumed it guaranteed.